1 - El Doctor Kosaraju es un no muy conocido, autodenominado "experto" en dietas. Acaba de crear la dieta ideal para el programador, llamada la "Zetadieta".
La propuesta busca resolver el problema central de que cada persona tendrá un objetivo diferente para la cantidad de carbohidratos, proteínas y grasas (en gramos) que desea ingerir diariamente. La zetadieta del Doctor Kosaraju es muy simple: ha elegido 3 "alimentos canónicos", y la dieta consistirá exclusivamente de esos 3 alimentos:

1. Banana (fuente de carbohidratos): Cada banana aporta 27 gramos de carbohidratos y 105 calorías.
2. Atún (fuente de proteínas): Cada lata de atún aporta 30 gramos de proteínas y 120 calorías.
3. Aceite de oliva (fuente de grasas): Cada gramo de aceite de oliva aporta 1 gramo de grasas y 9 calorías.

El doctor considera despreciable lo que aporta cada alimento a las otras categorías (por ejemplo, el atún tiene 0 carbohidratos y 0 grasas).
Debes escribir un programa que calcule la cantidad total de calorías que ingerirá diariamente una persona que sigue la zetadieta, sabiendo que desea ingerir cada día C gramos de carbohidratos, P gramos de proteínas y G gramos de grasas.
Las cantidades de bananas y de latas de atún a ingerir cada día serán las mínimas posibles que aseguren al menos C gramos de carbohidratos y al menos P gramos de proteínas, respectivamente (redondeando hacia arriba si es necesario para cubrir el objetivo). En cambio, el aceite de oliva se puede fraccionar y medir con precisión de gramos exactos para cubrir exactamente los G gramos de grasas.
Se debe implementar el programa que resuelva la zetadieta(C, P, G) que reciba tres enteros C, P, G con las cantidades en gramos de carbohidratos, proteínas y grasas respectivamente. La función debe retornar un único entero: la cantidad total de calorías.
EJEMPLOS:

- Ejemplo 1:
    - Entrada: C = 88, P = 90, G = 50
    - Salida: 1230 (Ya que se necesitan 4 bananas, 3 latas de atún y 50 gramos de aceite para cubrir los requisitos,dando un total de 4 * 105 + 3 * 120 + 50 * 9 = 1230 calorías).

ENTRADA DE EVALUACION:

- Entrada: C = 248, P = 196, G = 64
- Salida esperada: Un número

2 - En este desafío nos adentraremos en el mundo de los autómatas celulares y la evolución artificial. Imagina un arreglo unidimensional de N celdas, donde cada celda puede tener un estado de 1 (activa) o 0 (inactiva).
El sistema evoluciona a lo largo de E ciclos de tiempo. Las reglas del proceso son las siguientes:

1. Inicialización algorítmica: El arreglo inicial de tamaño N se genera mediante una condición matemática. Para cada posición i (desde 0 hasta N - 1), el valor inicial es 1 si la condición (i * 7 + 3) % 5 == 0 se cumple, y 0 en caso contrario.
2. Ciclos de evolución: Se deben simular E ciclos consecutivos. En cada ciclo, cada celda se actualiza basándose en sus vecinos inmediatos (su valor anterior, el de su izquierda y el de su derecha, utilizando condiciones de frontera circulares: el vecino a la izquierda de la celda 0 es la celda N - 1, y el vecino a la derecha de la celda N - 1 es la celda 0).
    - La regla de transición para el siguiente estado de la celda i es: si la suma de los valores de la celda i y sus dos vecinos (izquierda y derecha) en el ciclo anterior es un número par, la celda pasa a ser 1; si la suma es impar, pasa a ser 0.

Tu tarea es escribir un programa que, dados los valores de N y E, ejecute la carga inicial y los E ciclos de evolución descritos, y finalmente retorne la suma total de todos los valores del arreglo resultante.
Especificación del programa:

- Parámetros:
    1. N: Entero que indica la cantidad de celdas (tamaño del arreglo).
    2. E: Entero que indica la cantidad de ciclos de evolución a simular.
- Valor de retorno:
    1. Un número entero que representa la suma total de los valores de todas las celdas del arreglo tras completar los E ciclos.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 10, E = 3
    - Salida: 4

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 300, E = 72
- Formato de respuesta esperado en el sistema: Un número

3 - En este desafío nos adentraremos en el mundo de los autómatas celulares bidimensionales. Imagina una matriz de N filas por M columnas, donde cada celda puede tener un estado de 1 (activa) o 0 (inactiva).

El sistema evoluciona a lo largo de E ciclos de tiempo. Las reglas del proceso son las siguientes:

1. Inicialización algorítmica: La matriz se genera mediante una condición matemática basada en sus coordenadas.Para cada celda en la posición de la fila i y columna j (con i desde 0 hasta N-1 y j desde 0 hasta M-1), el valor inicial es 1 si la condición (i * 5 + j * 3 + 2) % 7 == 0 se cumple, y 0 en caso contrario.
2. Ciclos de evolución: Se deben simular E ciclos consecutivos. En cada ciclo, el estado de cada celda se actualiza basándose en la suma de los valores de sus 8 vecinos inmediatos (arriba, abajo, izquierda, derecha y las cuatro diagonales), utilizando condiciones de frontera toroidales (el tablero se conecta por los bordes, es decir, la fila superior se conecta con la inferior y la izquierda con la derecha).
    - La regla de transición para el siguiente estado de la celda (i, j) es: si la cantidad de vecinos activos (con valor 1) es un número par, la celda pasa a ser 1; si la cantidad de vecinos activos es impar, pasa a ser 0.(Nota: el valor de la celda propia no se cuenta dentro de sus vecinos para esta regla).

Tu tarea es escribir un programa que, dados los valores de N, M y E, ejecute la carga inicial en la matriz y los E ciclos de evolución descritos, y finalmente retorne la suma total de todos los valores de la matriz resultante.

Especificación del programa:

- Parámetros:
    1. N: Entero que indica la cantidad de filas de la matriz.
    2. M: Entero que indica la cantidad de columnas de la matriz.
    3. E: Entero que indica la cantidad de ciclos de evolución a simular.
- Valor de retorno:
    1. Un número entero que representa la suma total de los valores de todas las celdas de la matriz tras completar los E ciclos.

Ejemplos:

- Ejemplo 1:
    - Entrada: N = 5, M = 5, E = 3
    - Salida: 12

Para avanzar al siguiente problema, ingresa el resultado que arroja tu programa para la siguiente entrada de prueba oculta:

- Entrada de evaluación: N = 50, M = 50, E = 36
- Formato de respuesta esperado en el sistema: Un número

4 - En este desafío nos adentraremos en la teoría de números para identificar los esquivos números perfectos. Como recordarás, un número perfecto es aquel entero positivo que es igual a la suma de sus divisores propios positivos (excluyéndose a sí mismo). Tu tarea es escribir un programa que, dado un número entero límite N, encuentre todos los números perfectos que sean menores o iguales a N, y retorne la multiplicación total de todos ellos pero al total restarle la mitad del último encontrado.

Especificación del programa:

- Parámetros:
    1. N: Entero positivo que indica el límite máximo hasta donde se debe buscar.
- Valor de retorno:
    1. Un número entero